Embodied intelligent functions core source code

1.Course content

1. AI large model embodied intelligence is a composite function involving the coordination of multiple node programs. This section explains the core programs.

2.Source Code Package Structure

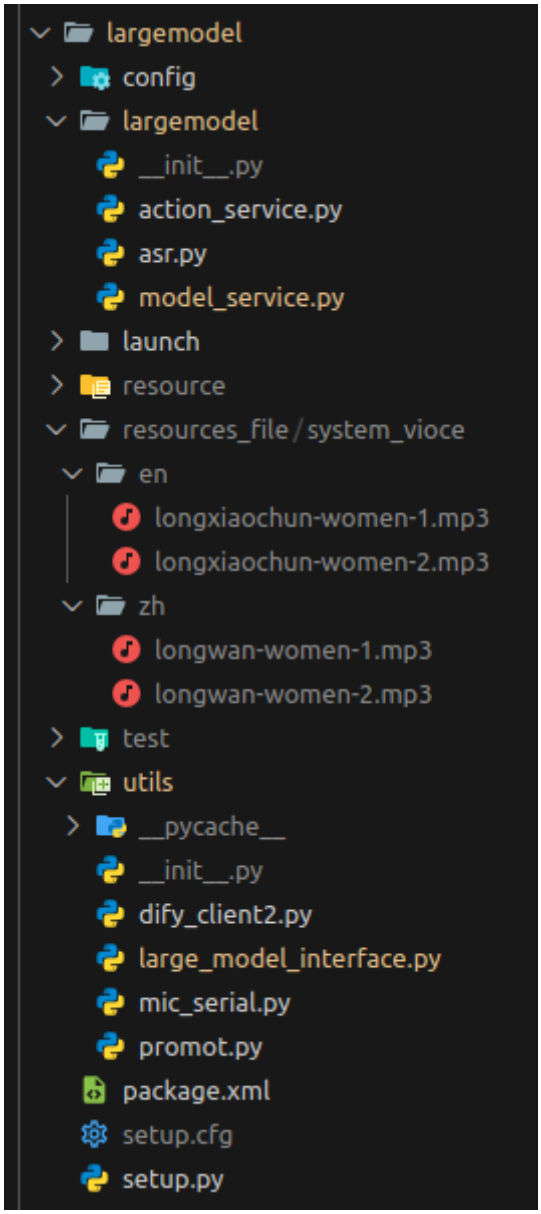
2.1 Package File Structure

The ROS package for AI large model embodied intelligence is largemodel, with the package path:

First enter docker

```
root/LargeModel_ws/src/largemodel
```

The package file structure is as follows:



Folder and file descriptions:

2.1.1 config

Configuration folder, containing configuration files

- large_model_interface.yaml

  Large model interface configuration file, used to configure API-KEYs and large model parameters for various platforms

- yahboom.yaml

  Node configuration file: used to configure core node parameters

2.1.2 largemodel

Source code folder, core programs for AI large model embodied intelligence

- asr.py

  Speech recognition program file, used to run speech recognition models and interact with users

- model_service.py

Model server program file, used to call various model interfaces to implement model inference architecture

- action_service.py

  Action server program file, used to receive action lists requested by the model server, control robot movement and play sounds

2.1.3 launch

Launch file folder, containing ROS2 node launch files

- largemodel_control.launch.py

  Launch file for AI large model embodied intelligence: starts multiple nodes, with two startup modes: voice interaction mode and text interaction mode

2.1.4 resources_file

Audio files for storing system sounds

2.1.5 utils

Component folder, containing non-core function program files

- large_model_interface.py

  Large model interface program file, containing underlying interfaces for calling large models from various platforms. Different platforms and models have different calling programs, which are uniformly managed in the interface file

- mic_serial.py

  Voice module driver program file, used to drive the wake word function of the voice module

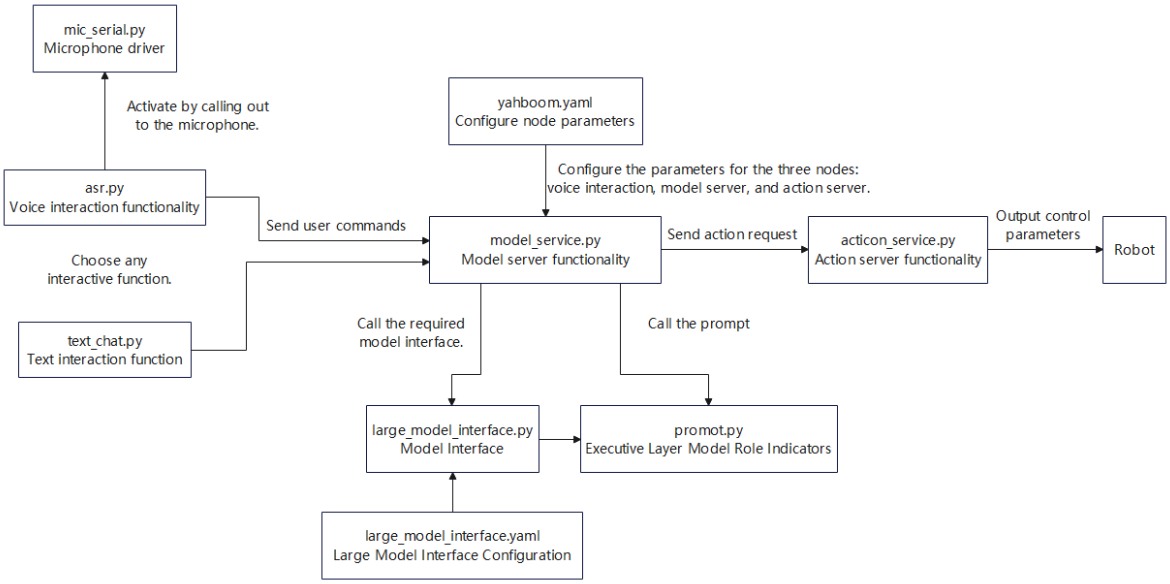
- promot.py

  Large model prompt file: used to generate prompts for the execution layer large model

- dify_client2.py

  dify API function, used for international version dify to request local dify applications

2.2 Inter-program Call Relationship Diagram



3.Speech Recognition Function

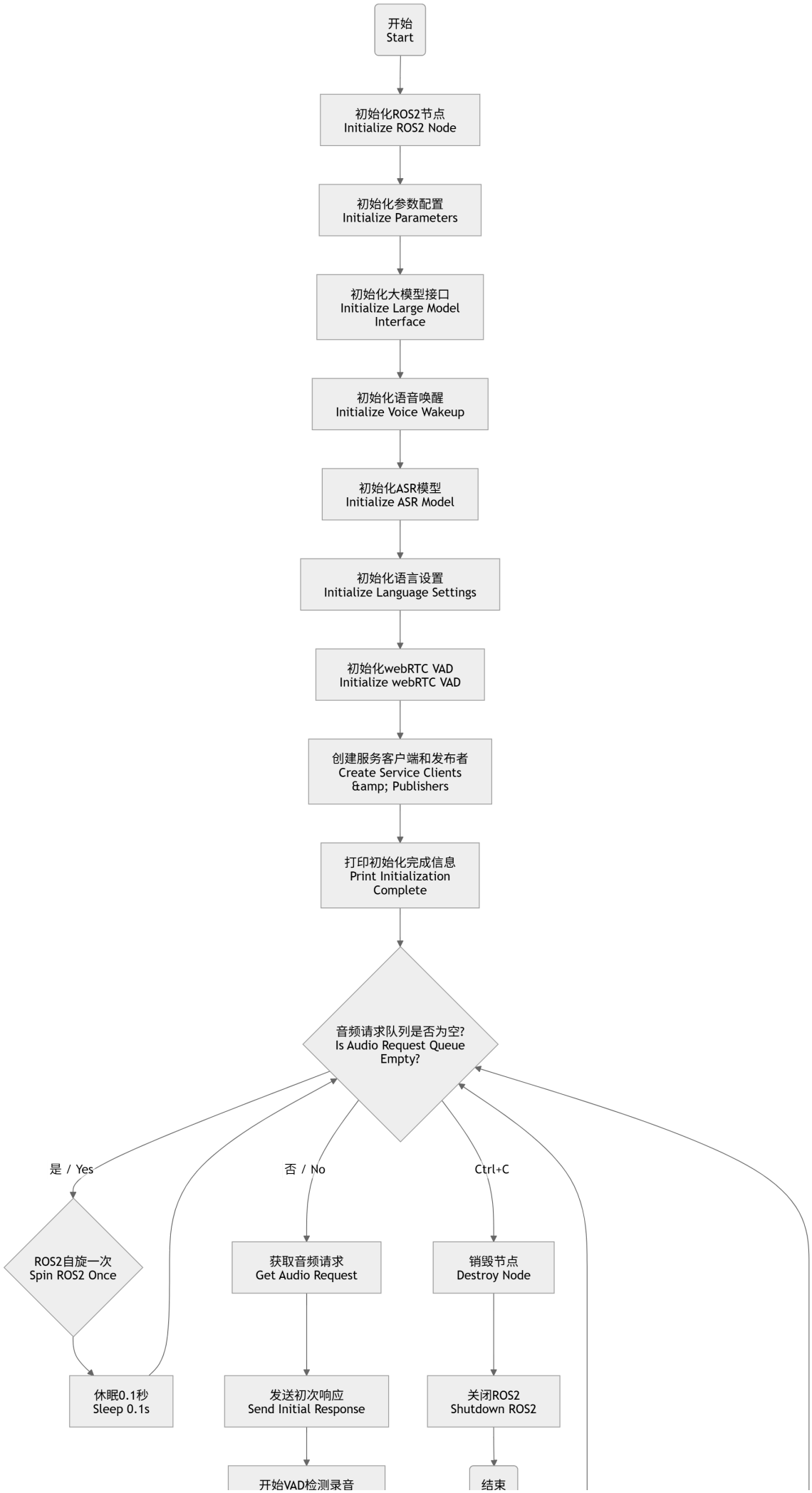
Speech recognition function includes VAD speech activity detection and speech-to-text conversion, source code path:

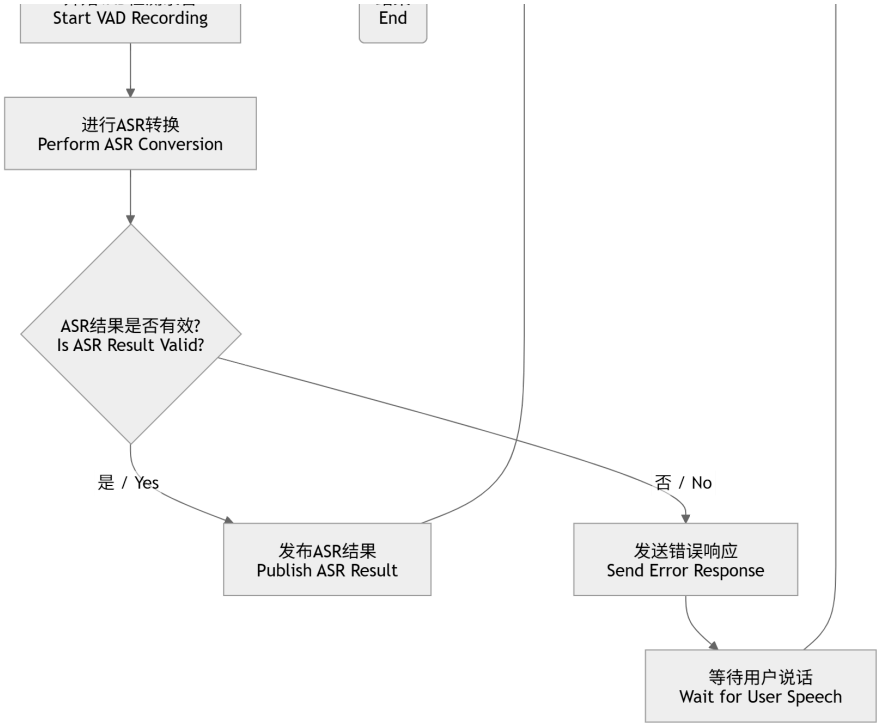
First enter docker

```
root/LargeModel_ws/src/largemodel/largemodel/asr.py
```

3.1 Program Flowchart

ASR program flowchart


开始
Start
初始化ROS2节点
Initialize ROS2 Node
初始化参数配置
Initialize Parameters
初始化大模型接口
Initialize Large Model Interface
初始化语音唤醒
Initialize Voice Wakeup
初始化ASR模型
Initialize ASR Model
初始化语言设置
Initialize Language Settings
初始化webRTC VAD
Initialize webRTC VAD
创建服务客户端和发布者
Create Service Clients & Publishers
打印初始化完成信息
Print Initialization Complete
音频请求队列是否为空?
Is Audio Request Queue Empty?
是 / Yes
否 / No
Ctrl+C
ROS2自旋一次
Spin ROS2 Once
获取音频请求
Get Audio Request
销毁节点
Destroy Node
休眠0.1秒
Sleep 0.1s
发送初次响应
Send Initial Response
关闭ROS2
Shutdown ROS2
开始VAD检测录音
结束



3.2 VAD Speech Activity Detection

Implementation: listen_for_speech method in ASRNode class

Program Analysis: Real-time recording through specified microphone, using VAD (Voice Activity Detection) to determine if speaking is occurring. When speech ends (continuous silence exceeds set frame count), stop recording and save valid speech content to file.

Detailed logic:

1. Initialize audio stream and configure parameters (such as sample rate, number of channels, etc.).
2. Continuously read audio frames and perform speech activity detection.
3. If speech start is detected, add audio frames to buffer; if continuous silence exceeds threshold (90 frames, about 1s), end recording.
4. After recording ends, remove trailing silence part and save valid speech as WAV file.
5. If no valid speech is detected, do not save file.

```
  def listen_for_speech(self, mic_index=0):
        p = pyaudio.PyAudio()
        audio_buffer = []
        silence_counter = 0
        MAX_SILENCE_FRAMES = 90  # 30帧*30ms=900ms静音后停止 / Stop after 900ms of
silence (30 frames * 30ms)
        speaking = False  # 语音活动标志 / Flag indicating speech activity
        frame_counter = 0  # 计数器 / Frame counter
        stream_kwargs = {
            "format": pyaudio.paInt16,
            "channels": 1,
            "rate": self.sample_rate,
            "input": True,
            "frames_per_buffer": self.frame_bytes,
        }
        if mic_index != 0:
            stream_kwargs["input_device_index"] = mic_index
```

```
        # 通过蜂鸣器提示用户讲话 / Prompt the user to speak via the buzzer
        self.pub_beep.publish(UInt16(data=1))
        time.sleep(0.5)
        self.pub_beep.publish(UInt16(data=0))

        try:
            # 打开音频流 / Open audio stream
            stream = p.open(**stream_kwargs)
            while True:
                if self.stop_event.is_set():
                    return False

                frame = stream.read(
                    self.frame_bytes, exception_on_overflow=False
                )  # 读取音频数据 / Read audio data
                is_speech = self.vad.is_speech(
                    frame, self.sample_rate
                )  # VAD检测 / VAD detection

                if is_speech:
                    # 检测到语音活动 / Detected speech activity
                    speaking = True
                    audio_buffer.append(frame)
                    silence_counter = 0
                else:
                    if speaking:
                        # 在语音活动后检测静音 / Detect silence after speech
activity
                        silence_counter += 1
                        audio_buffer.append(
                            frame
                        )  # 持续记录缓冲 / Continue recording buffer

                        # 静音持续时间达标时结束录音 / End recording when silence
duration meets the threshold
                        if silence_counter >= MAX_SILENCE_FRAMES:
                            break
                frame_counter += 1
                if frame_counter % 2 == 0:
                    self.get_logger().info("1" if is_speech else "-")
                    # print('1-' if is_speech else '0-', end='', flush=True)  #
实时状态显示方式 / Real-time status display
        finally:
            stream.stop_stream()
            stream.close()
            p.terminate()

        # 保存有效录音（去除尾部静音） / Save valid recording (remove trailing
silence)
        if speaking and len(audio_buffer) > 0:
            # 裁剪最后静音部分 / Trim the last silent part
            clean_buffer = (
                audio_buffer[:-MAX_SILENCE_FRAMES]
                if len(audio_buffer) > MAX_SILENCE_FRAMES
                else audio_buffer
            )

            with wave.open(self.user_speechdir, "wb") as wf:
```

```
                wf.setnchannels(1)
                wf.setsampwidth(p.get_sample_size(pyaudio.paInt16))
                wf.setframerate(self.sample_rate)
                wf.writeframes(b"".join(clean_buffer))
                return True
```

3.3 ASR Speech Recognition

Implementation: ASR_conversion method in ASRNode class converts recording files to text

Calls speech recognition model interface functions in large_model_interface.py large model interface file

Note: Text recognized by speech recognition with less than 4 characters will be considered invalid, to prevent content after false wake-up from being treated as valid content

Program Analysis:

1. If using online_asr, call corresponding method and check if result is valid text (length greater than 4), return recognized content if successful, otherwise log error and return 'error'.
2. Otherwise use SenseVoiceSmall_ASR for recognition, with same result judgment and processing.

   Where SenseVoiceSmall_ASR is a local model speech recognition method, only available on jetson orin nano, jetson orin NX hosts.

```
def ASR_conversion(self, input_file):
    if self.use_oline_asr:
        result=self.modelinterface.oline_asr(input_file)
        if result[0] == 'ok' and len(result[1]) > 4:
            return result[1]
        else:
            self.get_logger().error(f'ASR Error:{result[1]}')
            return 'error'
    else:
        result=self.modelinterface.SenseVoiceSmall_ASR(input_file)
        if result[0] == 'ok' and len(result[1]) > 4:
            return result[1]
        else:
            self.get_logger().error(f'ASR Error:{result[1]}')
            return 'error'
```

4.Model Server Function

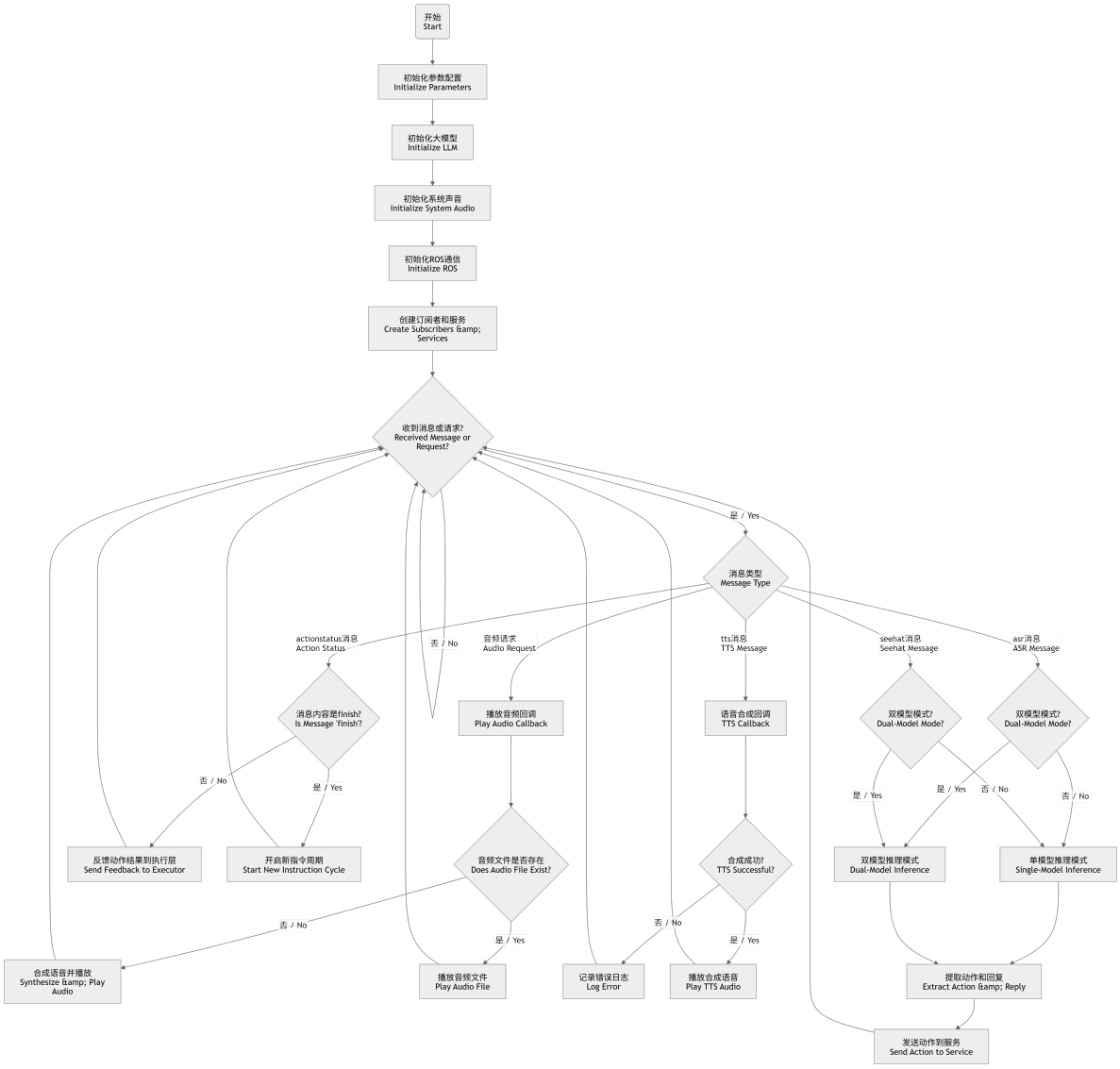
Implementation program is model_service.py, used to receive speech recognition results in voice interaction mode or terminal input in text interaction mode, implementing large model inference logic, source code path:

First enter docker

```
root/LargeModel_ws/src/largemodel/largemodel/model_service.py
```

4.1 Program Flowchart

model_service program flowchart



4.2 Dual Model Inference (Domestic Version)

Default uses dual model inference mode, implemented by dual_large_model_mode and instruction_process methods in LargeModelService class

Program Analysis:

1. If it's a new instruction cycle (self.new_order_cycle is True):
    - Initialize multimodal historical context;
    - Use task decision model to plan user input prompt;
    - Pass planning results to instruction execution layer model for processing;
    - Set new_order_cycle to False, indicating current instruction cycle starts execution.
2. Otherwise (non-new cycle):
    - Directly pass input prompt and type to instruction execution layer model for processing.

```
def dual_large_model_mode(self, type, prompt=""):
    """
    此函数实现了双模型推理模式，即先由文本生成模型进行任务规划，然后由多模态大模型生成动作列表
```

```
    This function implements the dual model inference mode, where the text
generation model first plans the task, and then the multimodal large model
generates the action list.
    """
    if (
        self.new_order_cycle
    ):  # 判断是否是新任务周期 / Determine if it is a new task cycle
        # 判断上一轮对话指令是否完成如果完成就清空历史上下文，开启新的上下文 / Determine if
the previous round of dialogue instructions are completed. If completed, clear
the historical context and start a new context
        self.model_client.init_Multimodal_history(
            get_prompt()
        )  # 初始化执行层上下文历史 / Initialize execution layer context history
        execute_instructions = self.model_client.TaskDecision(
            prompt
        )  # 调用决策层大模型进行任务规划 / Call the decision layer large model for
task planning

        if not execute_instructions == "error":
            if self.text_chat_mode:
                msg = String(
                    data=f"The upcoming task to be carried out:
{execute_instructions}"
                )
                self.text_pub.publish(msg)
            else:
                self.get_logger().info(
                    f"The upcoming task to be carried out:
{execute_instructions}"
                )  # 即将执行的任务: ... / Upcoming task to be executed...
            self.instruction_process(
                type="text",
                prompt=f"用户: {prompt},决策层AI规划:{execute_instructions}",
            )  # 传递决策层模型规划好的执行步骤给执行层模型 / Pass the planned execution
steps from the decision layer model to the execution layer model
            self.new_order_cycle = (
                False  # 重置指令周期标志位 / Reset the instruction cycle flag
            )
        else:
            self.get_logger().info(
                "The model service is abnormal. Check the large model account or
configuration options"
            )  # 模型推理失败，请检查模型配额和账户是否正常！！！ / Model inference
failed, please check the model quota and account whether normal!!!
    else:
        self.instruction_process(
            prompt, type
        )  # 调用执行层大模型生成成动作列表并执行 / Call the execution layer large
model to generate an action list and execute

def instruction_process(self, prompt, type, conversation_id=None):
    """
    根据输入信息的类型（文字/图片），构建不同的请求体进行推理，并返回结果）
    Based on the type of input information (text/image), construct different
request bodies for inference and return the result.
    """
    if self.regional_setting == "China":  # Domestic version
        if type == "text":
```

```
            raw_content = self.model_client.multimodalinfer(prompt)
        elif type == "image":
            self.save_single_image()
            raw_content = self.model_client.multimodalinfer(
                "机器人反馈:执行seewhat()完成", image_path=self.image_save_path
            )
        json_str = self.extract_json_content(raw_content)

    elif self.regional_setting == "international":  # International version
        if type == "text":
            result = self.model_client.TaskExecution(
                input=prompt,
                map_mapping=self.map_mapping,
                language=self.language_dict[self.language],
                conversation_id=conversation_id,
            )
            if result[0]:
                json_str = self.extract_json_content(result[1])
                self.conversation_id = result[2]
            else:
                self.get_logger().info(f"ERROR:{result[1]}")
        elif type == "image":
            self.save_single_image()
            result = self.model_client.TaskExecution(
                input="机器人反馈:执行seewhat()完成",
                map_mapping=self.map_mapping,
                language=self.language_dict[self.language],
                image_path=self.image_save_path,
                conversation_id=conversation_id,
            )
            if result[0]:
                json_str = self.extract_json_content(result[1])
                self.conversation_id = result[2]
            else:
                self.get_logger().info(f"ERROR:{result[1]}")

    if json_str is not None:
        # 解析JSON字符串,分离"action"、"response"字段 / Parse JSON string, separate
"action" and "response" fields
        action_plan_json = json.loads(json_str)
        action_list = action_plan_json.get("action", [])
        llm_response = action_plan_json.get("response", "")
    else:
        self.get_logger().info(
            f"LargeScaleModel return: {json_str},The format was unexpected. The
output format of the AI model at the execution layer did not meet the
requirements"
        )
        return

    if self.text_chat_mode:
        msg = String(data=f'"action": {action_list}, "response":
{llm_response}')
        self.text_pub.publish(msg)
    else:
        self.get_logger().info(
            f'"action": {action_list}, "response": {llm_response}'
        )
```

```
    self.send_action_service(
        action_list, llm_response
    )  # 异步发送动作列表、回复内容给ActionServer / Asynchronously send action list
and response content to ActionServer
```

4.3 Dual Model Inference (International Version)

1. Program implementation logic is the same as domestic version, difference is that international version requests local dify application, then dify requests cloud large model

```
def dual_large_model_international_model(self, type, prompt=""):
    """
    This function is applicable to international version dual model inference
mode, using dify as middleware
    """
    if (
        self.new_order_cycle
    ):  # 判断是否是新任务周期 / Determine if it is a new task cycle
        self.conversation_id = None
        result = self.model_client.TaskDecision(prompt)
        if result[0]:
            if self.text_chat_mode:  # 文字交互模式 / Text interaction mode
                msg = String(
                    data=f"The upcoming task to be carried out:{result[1]}"
                )
                self.text_pub.publish(msg)
            else:  # 语音交互模式 / Voice interaction mode
                self.get_logger().info(
                    f"The upcoming task to be carried out:{result[1]}"
                )

            self.instruction_process(
                type="text", prompt=f"用户：{prompt},决策层AI规划:{result[1]}"
            )
            self.new_order_cycle = (
                False  # 重置指令周期标志位 / Reset the instruction cycle flag
            )
        else:
            self.get_logger().info(f"ERROR: {result[1]}")

    else:
        self.instruction_process(
            prompt, type, conversation_id=self.conversation_id
        )  # 调用执行层大模型生成成动作列表并执行 / Call the execution layer large
model to generate an action list and execute
```

5.Action Server Function

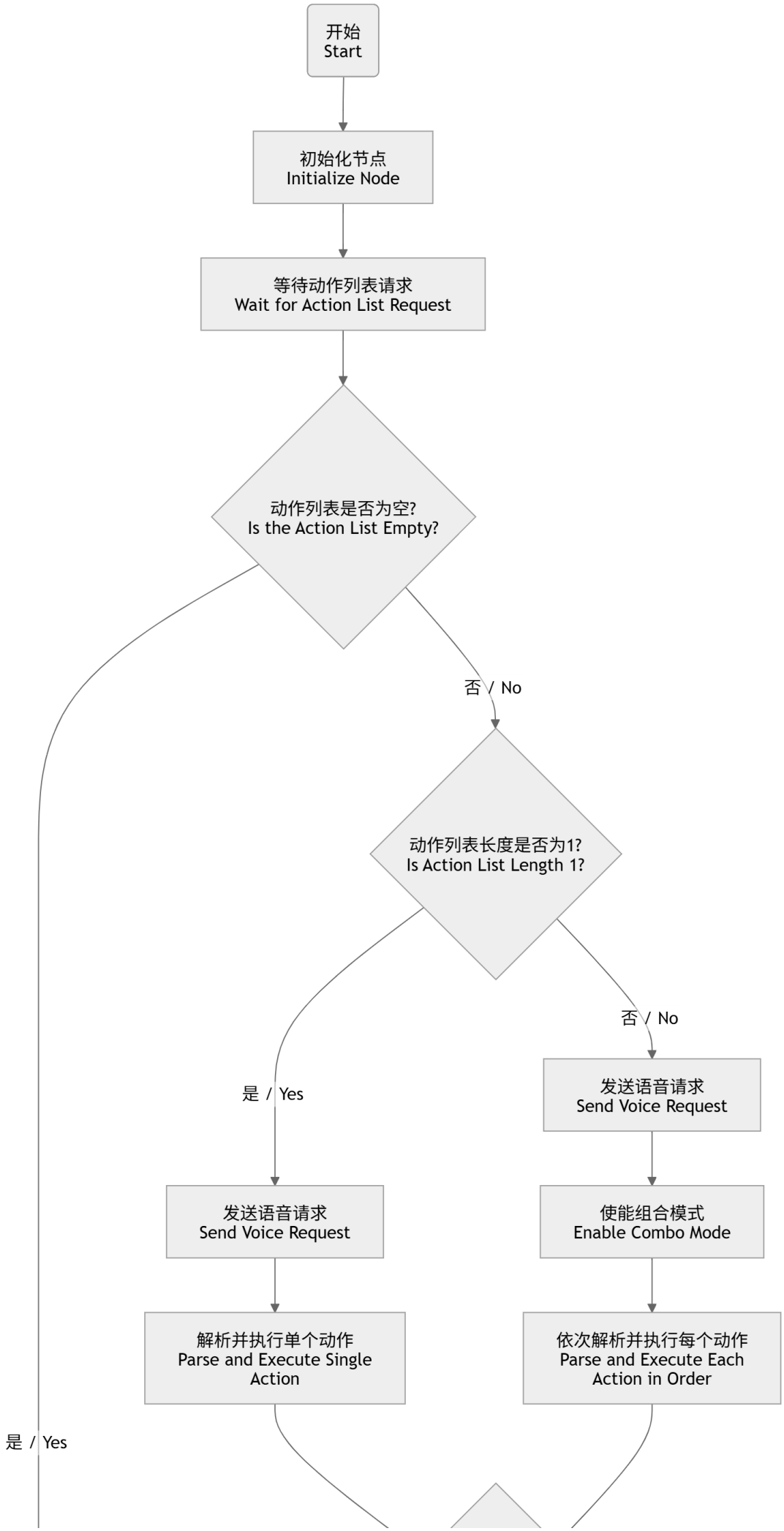
Implementation program is action_service.py, used to receive action lists requested by model server, parse action lists and execute them, source code path:

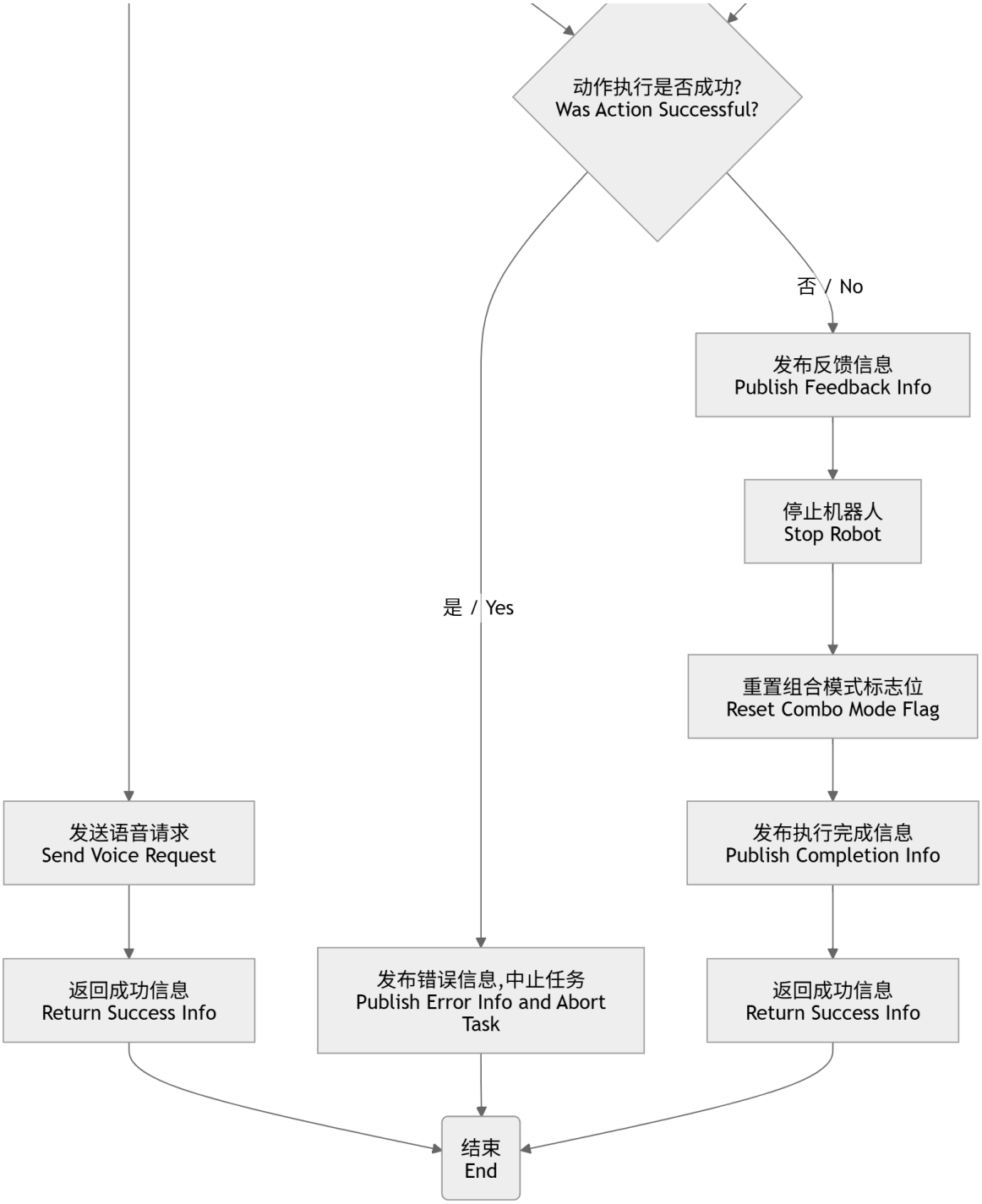
First enter docker

```
root/LargeModel_ws/src/largemodel/largemodel/action_service.py
```

5.1 Program Flowchart

Saction_service program flowchart


开始
Start
初始化节点
Initialize Node
等待动作列表请求
Wait for Action List Request
动作列表是否为空?
Is the Action List Empty?
否 / No
是 / Yes
动作列表长度是否为1?
Is Action List Length 1?
是 / Yes
否 / No
发送语音请求
Send Voice Request
发送语音请求
Send Voice Request
使能组合模式
Enable Combo Mode
解析并执行单个动作
Parse and Execute Single Action
依次解析并执行每个动作
Parse and Execute Each Action in Order



5.2 Action Function Library

Robot action functions come from methods in CustomActionServer class, containing various sub-action functions. Here we use the robot arm upward control action function as an example. Other action functions will be explained when they first appear in subsequent chapters

Program Analysis:

1. Convert input string parameters to floating point numbers
2. Directly call function, pass parameters, communicate with underlying driver board, control six servos
3. Decide whether to publish completion information based on mode

```
def arm_up(self):  # Robot arm upward
    #Call function, directly communicate with underlying driver board, control six servos to extend upward
    Arm.Arm_serial_servo_write6(90,90,90,90,90,90,1000)
    #If not in combination mode and not interrupted, publish completion information for controlling robot arm upward
    if not self.combination_mode and not self.interrupt_flag:
        self.action_status_pub("arm_up_done")
```

5.3 Parse Action List to Control Robot Function

Parse action lists generated by large model into functions that control robot entities, and execute them. Implemented by execute_callback method in CustomActionServer class

Program Analysis:

1. Receive action list sent by client (string format);
2. If action list is empty, directly return success result;
3. If only one action, parse and execute corresponding method, abort if failed;
4. If multiple actions, enter combination mode and execute sequentially, recording logs and feedback during process;
5. After all actions are executed, call stop() to stop robot;
6. Finally return execution success result to client.

```
def execute_callback(self, goal_handle):
    feedback_msg = Rot.Feedback()
    stop = getattr(self, "stop")  # Get stop method, stop robot motion by publishing topic
    actions=goal_handle.request.actions
    if not actions: # If action list is empty, directly return success
        if not self.text_chat_mode:
           self.send_Audiorequest(goal_handle.request.llm_response)

        if self.use_double_llm:
            self.action_status_pub('机器人反馈：回复用户完成')

        goal_handle.succeed()
        result = Rot.Result()
        result.success = True
        return result
    elif len(actions) == 1:# If action list has only one action, normal execution flow
        # Case 2: List has only one element
        action = actions[0]
        if not self.text_chat_mode:
           self.send_Audiorequest(goal_handle.request.llm_response)

        match = re.match(r"(\w+)\((.*)\)", action)
        if not match:
            self.get_logger().warning(f'action_service: {action} is invalid action,skip execution')
            goal_handle.abort()
            result = Rot.Result()
            result.success = False
            return result
        else:
```

```
            action_name, args_str = match.groups()
            args = [arg.strip() for arg in args_str.split(",")] if args_str else
[]
            try:
                if hasattr(self, action_name):
                    method = getattr(self, action_name)
                    method(*args)
                    feedback_msg.status = f'action service execute  {action}
successed'
                    # self.get_logger().info(feedback_msg.status)  # 添加日志
                    goal_handle.publish_feedback(feedback_msg)
                else:
                    self.get_logger().warning(f'action_service:inviald:
{action_name},skip execution')
                    self.action_status_pub('机器人反馈:动作函数不存在，无法执行')
                    goal_handle.abort()
                    result = Rot.Result()
                    result.success = False
                    return result
            except Exception as e:
                self.get_logger().error(f'action_service:execute action:
{action} failed,error-log: {str(e)}')
                goal_handle.abort()
                result = Rot.Result()
                result.success = False
                return result
    else:#If action list has multiple actions, enable combination mode
        if not self.text_chat_mode:
           self.send_Audiorequest(goal_handle.request.llm_response)

        self.combination_mode=True
        for action in actions:
            # Use regular expression to parse action name and parameters
            match = re.match(r"(\w+)\((.*)\)", action)
            if not match:
                self.get_logger().warning(f'action_service: {action} is invalid
action, skip execution')
                continue

            action_name, args_str = match.groups()
            args = [arg.strip() for arg in args_str.split(",")] if args_str else
[]

            try:

                # Check if action is a method in CustomActionServer class
                if hasattr(self, action_name):
                    method = getattr(self, action_name)
                    # Dynamically call method and pass parameters
                    method(*args)
                    # method(*converted_args)
                    feedback_msg.status = f'action service execute  {action}
successed'
                    self.get_logger().info(feedback_msg.status)  # Add log
                    goal_handle.publish_feedback(feedback_msg)
                else:
                    self.get_logger().warning(f'action_service:inviald:
{action_name}, skip execution')
```

```
            except Exception as e:
                self.get_logger().error(f'action_service:execute action:
{action} failed,error-log: {str(e)}')
                goal_handle.abort()
                result = Rot.Result()
                result.success = False
                return result

        self.action_status_pub(f'机器人反馈：执行{actions}完成')
        self.combination_mode=False#Reset combination mode flag
    stop()  # Execute all actions to stop robot
    # Return success information to client
    goal_handle.succeed()
    result = Rot.Result()
    result.success = True
    return result
```

6.Interrupt Function

Robot supports interruption at any stage, specifically can be divided into recording stage interruption, dialogue stage interruption, action stage interruption. Here we introduce the principle of interruption at each stage

6.1 Recording Stage Interruption

If during recording process you find you said something wrong, or are not satisfied with the speaking content and need to re-record, during **the recording process** you can directly wake up again to interrupt the previous recording and directly restart speech recording

- Logic implementation is main_loop method of **ASRNode** class in asr.py file:
- Each time when waking up, if there is already a running wake-up recording thread, interrupt it through thread event `stop_event` and wait for it to end;
- After clearing the stop event flag, start a new recording thread

```
def main_loop(self):
    while rclpy.ok():
        while (
            self.audio_request_queue.qsize() > 1
        ):  # 只处理最近的一次唤醒请求，防止重复唤醒 / Process only the most recent
wake-up request to prevent duplicates
            self.audio_request_queue.get()

        if not self.audio_request_queue.empty():
            self.audio_request_queue.get()
            self.wakeup_pub.publish(
                Bool(data=True)
            )  # 发布唤醒信号 / Publish wake-up signal
            self.get_logger().info("I'm here")
            playsound(
                self.audio_dict[self.first_response]
            )  # 应答用户 / Respond to the user

            if (
                self.current_thread and self.current_thread.is_alive()
            ):  # 打断上次的唤醒处理线程 / Interrupt the previous wake-up handling
thread
```

```
                self.stop_event.set()
                self.current_thread.join()  # 等待当前线程结束 / Wait for the current thread to finish
                self.stop_event.clear()  # 清除事件 / Clear the event
            self.current_thread = threading.Thread(target=self.kws_handler)
            self.current_thread.daemon = True
            self.current_thread.start()
        rclpy.spin_once(self, timeout_sec=0.1)
        time.sleep(0.1)
```

6.2 Dialogue Stage Interruption

If during robot speaking process, you feel the robot's response is unsatisfactory or don't want the robot to continue responding, you can use the wake word to directly interrupt the robot's speech, directly start speech recording. At this time you can give new instructions to the robot (still in current task cycle), or say "end current task" to directly end the current task and start a new task cycle.

- Logic implementation is wakeup_callback and play_audio methods of **CustomActionServer** class in action_service.py file:
- wakeup_callback is the callback function for wake-up handling. Each time when waking up in asr.py program, wake-up signal will be published through topic communication, wakeup_callback subscribes to and processes the signal.
- Each time when waking up, detect whether **pygame.mixer** is playing audio, if so, notify the playing thread to stop playing through thread event self.stop_event
- If after wake-up it's detected that previous action is running, set **self.interrupt_flag** flag bit, used for subsequent action interruption

```
def wakeup_callback(self, msg):
    if msg.data:
        if pygame.mixer.music.get_busy():
            self.stop_event.set()
        if self.action_runing:
            self.interrupt_flag = True
            self.stop()
            self.pubSix_Arm(self.init_joints)
```

play_audio播放音频时会检测self.stop_event是否被置位，一旦检测到置位，立刻停止当前播放的音频。

```
def play_audio(self, file_path: str, feedback: Bool = False) -> None:
    """
    同步方式播放音频函数The function for playing audio in synchronous mode
    """
    pygame.mixer.music.load(file_path)
    pygame.mixer.music.play()
    while pygame.mixer.music.get_busy():
        if self.stop_event.is_set():
            pygame.mixer.music.stop()
            self.stop_event.clear()  # Clear event
            return
        pygame.time.Clock().tick(10)
    if feedback:
```

```
        self.action_status_pub("机器人反馈：回复用户完成")
```

6.3 Action Stage Interruption

Robot will stop current action and restore initial posture when woken up during action execution.

For example, actions like robot arm grabbing, sorting machine codes, etc. need to start external programs in subprocesses. Here we use the color block moving function change_pose as an example:

When color block moving is not completed, it will keep waiting in the while not self.change_pose_future.done(): loop: during this process, if it detects that the self.interrupt_flag interruption flag is set, it will first call check_close_change_pose function to recursively end the subprocess tree, then control the robot arm to return to grabbing posture and end the task.

```
def change_pose(self, x1, y1, x2, y2,src,tar,side,tar_size):
    src_color = src.strip("'\"")  # Remove single quotes and double quotes
    self.cur_down_name = src_color
    self.cur_down_pose[self.cur_down_name] = []
    cmd1 = "ros2 run largemodel_arm Change_Pose"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=Change_Pose",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )
    time.sleep(8.0)
    x1 = int(x1)
    y1 = int(y1)
    x2 = int(x2)
    y2 = int(y2)
    side = int(side)
    tar_size = int(tar_size)
    #time.sleep(5.0)
    #self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2]))

    while not self.object_position_pub.get_subscription_count():
        self.get_logger().info("--------------------continue publish------------
------")
        self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2,
y2,side,tar_size]))
        time.sleep(0.1)
    self.get_logger().info("--------------------Publish Int16Array done---------
---------")
    self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2,
y2,side,tar_size]))

    while not self.change_pose_future.done():
        if self.interrupt_flag:
            self.check_close_change_pose()
            #self.pubSix_Arm(self.init_joints)  # Robot arm retract
```

```
            Arm.Arm_serial_servo_write6(90,120,0,0,90,30,1000)
            return
        time.sleep(0.1)

    result = self.change_pose_future.result()
    self.check_close_change_pose()
    if not self.interrupt_flag:
        if result.data == "change_pose_done":
            self.action_status_pub("change_pose_done", x1=x1, y1=y1, x2=x2,
y2=y2)
        else:
            self.action_status_pub("change_pose_failed", x1=x1, y1=y1, x2=x2,
y2=y2)

    self.change_pose_future = Future()  # Reset Future object
    if self.interrupt_flag:
        time.sleep(0.5)
        Arm.Arm_serial_servo_write6(90,120,0,0,90,30,1000)
```